



Pautas de resolución y entrega

- Lea detenidamente el enunciado. Desarrolle la solución en BlueJ.
- Para la entrega, debe exportar el proyecto con su nombre y apellido, por ejemplo: **juanMartinez.zip** y debe subirlo en la tarea correspondiente en el entorno Asignaturas.

Enunciado

Movistar Arena posee información de los próximos 30 espectáculos del 2026. De cada espectáculo se conoce: código de espectáculo, fecha (DD/MM/AAAA), si es con campo o no, cantidad de localidades disponibles, precio inicial de las entradas y la información de la productora: nombre de la productora, cantidad de sponsors y si es extranjera o no.

Existen diferentes tipos de espectáculos:

- **Banda**, este tipo de espectáculo registra, el nombre de la banda y los integrantes que la componen (a lo sumo 8). De cada uno de estos se conoce: nombre y rol que ocupa (Baterista, Vocalista, Guitarrista, etc.).
- **Solista**, este tipo de espectáculo registra, la cantidad de canciones y el detalle del setlist (el listado de los nombres de las canciones que componen el show).

Se pide:

a) Implementar la clase **Espectáculo** con sus respectivos atributos, constructores y métodos para acceder y modificar sus atributos (*respecto a los métodos de acceso sólo implemente los que vaya a utilizar*). Además de todos los elementos necesarios para manejar los diferentes tipos de espectáculos.

b) Movistar Arena necesita saber cuál es el promedio de ganancia entre todos sus espectáculos. La ganancia que recibe el Movistar Arena se determina de la siguiente manera:

- En el caso de la **Banda**, la ganancia del espectáculo se calcula como el precio inicial de las entradas por la cantidad de localidades disponibles. Si el espectáculo incluye campo, a la cantidad de localidades disponibles hay que sumarle 8.000 lugares más. Por otro lado, si la cantidad de integrantes de la banda supera las 5 personas, al cálculo de la ganancia se le adhiere un 20% más por técnica de escenario. De toda la ganancia total, el Movistar Arena se queda con el 50%.
- En el caso del **Solista**, al igual que la Banda, la ganancia del espectáculo se calcula como el precio inicial de las entradas por la cantidad de localidades disponibles. Si el espectáculo incluye campo, a la cantidad de localidades disponibles hay que sumarle 10.000 lugares más. Por otro lado, si el setlist tiene entre 25 y 28 canciones, al cálculo de la ganancia se le adhiere un 15,5% más, mientras que si el setlist tiene más de 28 canciones se le suma un 21% más, por tiempo de show. De toda la ganancia total, el Movistar Arena se queda con el 42%.
- Para ambos casos, si el espectáculo está programado para los días: 24 de Marzo de 2026, 25 de Mayo de 2026, o 9 de Julio de 2026 se deberá pagar un extra del 10% sobre la ganancia que se lleva el Movistar Arena por ser feriados en Argentina. Además, si la productora tiene más de 5 sponsors debe pagarle un 12,5% más sobre la ganancia del Movistar Arena. Por otra parte, si la productora es nacional tiene un descuento del 10% sobre la ganancia que se lleva el Movistar Arena.

**Para ambos tipos de espectáculos, si se cumplen las condiciones, los aumentos/descuentos son acumulables.*

- c) Implementar los métodos necesarios para imprimir toda la información del Movistar Arena y sus espectáculos.
- d) Implementar un programa principal que instancie el Movistar Arena, agregue los espectáculos y se evalúen todos los casos (*no es necesario leer la información desde teclado*). Después de la carga, hacer la impresión de los datos utilizando lo implementado en el inciso c) y, por último, informar la ganancia promedio.

Clase Productora

```
public class Productora {
    private String nombre;
    private int cantSponsors;
    private boolean extranjera;

    public Productora(String nombre, int cantSponsors, boolean extranjera) {
        this.nombre = nombre;
        this.cantSponsors = cantSponsors;
        this.extranjera = extranjera;
    }

    public String getNombre() { return nombre; }
    public int getCantSponsors() { return cantSponsors; }
    public boolean isExtranjera() { return extranjera; }

    @Override
    public String toString() {
        return "Productora: " + nombre + " | Sponsors: " + cantSponsors + " | Extranjera: " + (extranjera ? "Si" : "No");
    }
}
```

datos Privados
Para
Encapsular

Constructor

getters

Clase Integrante

```
public class Integrante {
    private String nombre;
    private String rol;

    public Integrante(String nombre, String rol) {
        this.nombre = nombre;
        this.rol = rol;
    }

    public String getNombre() { return nombre; }
    public String getRol() { return rol; }

    @Override
    public String toString() {
        return nombre + " (" + rol + ")";
    }
}
```

Clase auxiliar para modelar a cada miembro

Construct.

getters

Clase Espectáculo <<Abstract>>

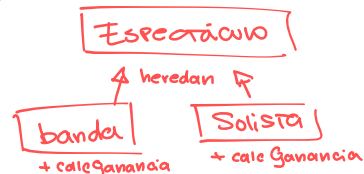
```
public abstract class Espectaculo {
    private String codigo;
    private String fecha; // Formato DD/MM/AAAA
    private boolean conCampo;
    private int localidadesDisponibles;
    private double precioInicial;
    private Productora productora;

    public Espectaculo(String codigo, String fecha, boolean conCampo, int localidadesDisponibles, double precioInicial,
        Productora productora) {
        this.codigo = codigo;
        this.fecha = fecha;
        this.conCampo = conCampo;
        this.localidadesDisponibles = localidadesDisponibles;
        this.precioInicial = precioInicial;
        this.productora = productora;
    }

    public String getCodigo() { return codigo; }
    public String getFecha() { return fecha; }
    public boolean isConCampo() { return conCampo; }
    public int getLocalidadesDisponibles() { return localidadesDisponibles; }
    public double getPrecioInicial() { return precioInicial; }
}
```

Atributos 3
Comparten las
Subclases
Banda y Solista

→ Para acceder a datos del obj. Productora



```

public Productora getProductora() { return productora; }

// Método abstracto para la ganancia del espectáculo según su tipo
public abstract double calcularGananciaEspectaculo(); Tanto Banda como Solista deben calcular su ganancia

// Método para calcular la ganancia que recibe el Movistar Arena
public double calcularGananciaMovistarArena() {
    double gananciaBaseMovistar = calcularGananciaEspectaculo();
    double porcentajeExtra = 0.0; → acumulador guarda % de aumento o descuento

    // Feriados nacionales en Argentina (24/03/2026, 25/05/2026, 09/07/2026)
    if (fecha.equals("24/03/2026") || fecha.equals("25/05/2026") || fecha.equals("09/07/2026")) { comparo String
        porcentajeExtra += 0.10; → suma un 10%
    }
    // Más de 5 sponsors
    if (productora.getCantSponsors() > 5) {
        porcentajeExtra += 0.125; → suma 12.5 %
    }
    // Descuento si la productora es nacional (no extranjera)
    if (!productora.isExtranjera()) {
        porcentajeExtra -= 0.10; → resto 10%
    }
    return gananciaBaseMovistar * (1.0 + porcentajeExtra); → Vct ganancia Final con el porcentaje acumulado
    = ganancia + (ganancia x Porcentaje) → Solo Factorice
}

@Override
public String toString() {
    return "Código: " + codigo + " | Fecha: " + fecha + " | Campo: " + (conCampo ? "Sí" : "No") +
        " | Localidades: " + localidadesDisponibles + " | Precio Base: $" + precioInicial +
        "\n" + productora.toString();
}
}

```

Clase Banda

```

public class Banda extends Espectaculo { Subclase Banda hereda de Espectaculo
    private String nombreBanda;
    private Integrante[] integrantes; → Vct. de objetos "Integrante" (max 8)
    private int cantIntegrantes; → dimension Lógica

    public Banda(String codigo, String fecha, boolean conCampo, int localidadesDisponibles, double precioInicial, Productora productora, String nombreBanda) {
        invoco constructor de Espectaculo
        super(codigo, fecha, conCampo, localidadesDisponibles, precioInicial, productora);
        this.nombreBanda = nombreBanda;
        this.integrantes = new Integrante[8]; // Max 8
        this.cantIntegrantes = 0;
    }

    public void agregarIntegrante(Integrante i) { agregar integrante al vector.
        if (cantIntegrantes < 8) {
            integrantes[cantIntegrantes] = i;
            cantIntegrantes++;
        }
    }

    public String getNombreBanda() { return nombreBanda; }
    public int getCantIntegrantes() { return cantIntegrantes; }

    @Override → Solista tiene el mismo método
    public double calcularGananciaEspectaculo() {
        int localidades = getLocalidadesDisponibles();
        if (isConCampo()) {
            localidades += 8000;
        }

        double gananciaTotal = getPrecioInicial() * localidades;
    }
}

```

```

        if (cantIntegrantes > 5) {
            gananciaTotal *= 1.20; // 20% más por técnica de escenario
        }
        // Movistar Arena se queda con el 50%
        return gananciaTotal * 0.50;
    }

    @Override
    public String toString() {
        String info = super.toString() + "\n Tipo: Banda | Nombre: " + nombreBanda + " | Integrantes (" + cantIntegrantes + "): ";
        for (int i = 0; i < cantIntegrantes; i++) {
            info += integrantes[i].toString() + (i < cantIntegrantes - 1 ? ", " : "");
        }
        return info;
    }
}

```

→ concateno datos Banda al toString de Espectaculo

Clase Solista

```

public class Solista extends Espectaculo {
    private String[] setlist; → lista con nom. canciones
    private int cantCanciones; → dim L

    public Solista(String codigo, String fecha, boolean conCampo, int localidadesDisponibles, double precioInicial, Productora productora, int maxCanciones) {
        super(codigo, fecha, conCampo, localidadesDisponibles, precioInicial, productora);
        this.setlist = new String[maxCanciones];
        this.cantCanciones = 0;
    }

    public void agregarCancion(String cancion) { → agrego cancion a lista
        if (cantCanciones < setlist.length) {
            setlist[cantCanciones] = cancion;
            cantCanciones++;
        }
    }

    public int getCantCanciones() { return cantCanciones; }

    @Override
    public double calcularGananciaEspectaculo() {
        int localidades = getLocalidadesDisponibles();
        if (isConCampo()) {
            localidades += 10000;
        }

        double gananciaTotal = getPrecioInicial() * localidades;

        if (cantCanciones >= 25 && cantCanciones <= 28) {
            gananciaTotal *= 1.155; // 15.5% extra por tiempo de show
        } else if (cantCanciones > 28) {
            gananciaTotal *= 1.21; // 21% extra por tiempo de show
        }

        // Movistar Arena se queda con el 42%
        return gananciaTotal * 0.42;
    }

    @Override
    public String toString() {
        return super.toString() + "\n Tipo: Solista | Cantidad de Canciones en Setlist: " + cantCanciones;
    }
}

```

Clase MovistarArena

```

public class MovistarArena { CLASE CONTIENE
    private Espectaculo[] espectaculos; 30 ESPECTÁCULOS ≠ MAIN
}

```

```

private int dimL;

public MovistarArena() {
    this.espectaculos = new Espectaculo[30]; // Próximos 30 espectáculos
    this.dimL = 0;
}

public void agregarEspectaculo(Espectaculo e) {
    if (dimL < 30) {
        espectaculos[dimL] = e;
        dimL++;
    }
}

public double calcularPromedioGanancia() {
    if (dimL == 0) return 0.0; → EVITO DIV. POR 0. ¡HÁGANLO! → buena práctica no a esta nada.
    double suma = 0.0;
    for (int i = 0; i < dimL; i++) {
        suma += espectaculos[i].calcularGananciaMovistarArena(); → Sum. ganancia final de cada espectáculo.
    }
    return suma / dimL; → Promedio
}

public void imprimirInformacion() { Método para imprimir toda la info (el System.out.println en el main)
    System.out.println("===== INFORMACIÓN MOVISTAR ARENA =====");
    System.out.println("Cantidad de Espectáculos Registrados: " + dimL);
    System.out.println("-----");
    for (int i = 0; i < dimL; i++) {
        System.out.println("Espectáculo #" + (i + 1) + ":");
        System.out.println(espectaculos[i].toString());
        System.out.println("Ganancia Movistar Arena: $" + String.format("%.2f",
espectaculos[i].calcularGananciaMovistarArena());
        System.out.println("-----");
    }
}
}

```

Clase Programa Principal

```

public class ProgramaPrincipal {
    public static void main(String[] args) {
        MovistarArena arena = new MovistarArena();

        // Productoras de prueba
        Productora prodNacionalPocosSponsors = new Productora("PopArt", 3, false);
        Productora prodExtranjeraMuchosSponsors = new Productora("Live Nation", 6, true);

        // Caso 1: Banda, con campo, >5 integrantes, fecha feriado (25/05/2026), productora nacional
        Banda banda1 = new Banda("ESP-001", "25/05/2026", true, 4000, 15000.0, prodNacionalPocosSponsors, "Los Auténticos
Decadentes");
        for (int i = 1; i <= 6; i++) {
            banda1.agregarIntegrante(new Integrante("Integrante " + i, "Músico"));
        }
        arena.agregarEspectaculo(banda1);

        // Caso 2: Solista, sin campo, setlist de 26 canciones, fecha normal, productora extranjera con >5 sponsors
        Solista solista1 = new Solista("ESP-002", "12/10/2026", false, 12000, 20000.0, prodExtranjeraMuchosSponsors, 30);
        for (int i = 1; i <= 26; i++) {
            solista1.agregarCancion("Canción " + i);
        }
        arena.agregarEspectaculo(solista1);

        // Caso 3: Solista, con campo, setlist de 30 canciones (>28), fecha feriado (09/07/2026), productora nacional
        Solista solista2 = new Solista("ESP-003", "09/07/2026", true, 5000, 18000.0, prodNacionalPocosSponsors, 35);
        for (int i = 1; i <= 30; i++) {
            solista2.agregarCancion("Tema " + i);
        }
    }
}

```

```
arena.agregarEspectaculo(solista2);

// Imprimir información completa → todos los datos
arena.imprimirInformacion();

// Informar la ganancia promedio
System.out.println("Ganancia promedio del Movistar Arena por espectáculo: $" +
    String.format("%.2f", arena.calcularPromedioGanancia()));
}
}
```
